IMPERIAL

Department of Chemical Engineering

Machine Learning for Chemical Engineering

Lecture 1b – Supervised Learning: Linear Regression

Data pre-processing (standardization)

Data pre-processing (standardization)

We have an example with 151 points in 2-dimensions $\mathbf{x} \in \mathbb{R}^2$

$$\mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(151)} & x_2^{(151)} \end{bmatrix}, \qquad \mathbf{X} \in \mathbb{R}^{151\times 2}$$

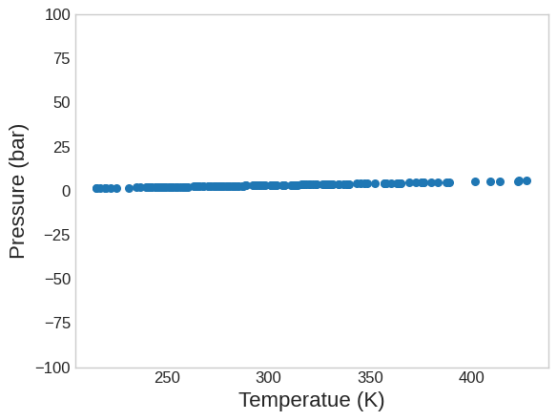
First step for any ML analysis (good practice) is **standardizing** the data:

$$\bar{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

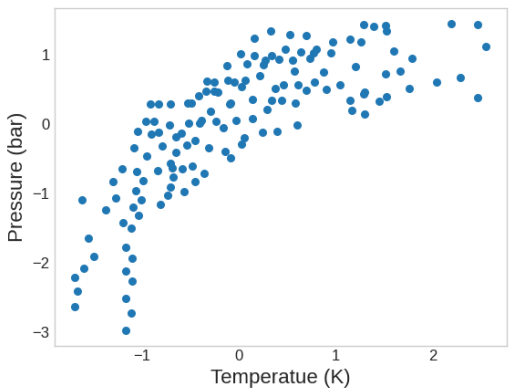
With

$$\mu_j = \frac{1}{151}\sum_{i=1}^{151} x_j^{(i)}, \qquad \sigma_j = \sqrt{\frac{\sum_{i=1}^{151}\left(x_j^{(i)} - \mu_j\right)^2}{150}}$$

Without standardization



With standardization



Data pre-processing (standardization)

We have an example with 151 points in 2-dimensions $\mathbf{x} \in \mathbb{R}^2$

$$\mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(151)} & x_2^{(151)} \end{bmatrix}, \qquad \mathbf{X} \in \mathbb{R}^{151\times 2}$$

First step for any ML analysis (good practice) is **standardizing** the data:

$$\bar{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

With

$$\mu_j = \frac{1}{151}\sum_{i=1}^{151} x_j^{(i)}, \qquad \sigma_j = \sqrt{\frac{\sum_{i=1}^{151}\left(x_j^{(i)} - \mu_j\right)^2}{150}}$$

$$\bar{\mathbf{X}} = \begin{bmatrix} \left(\bar{\mathbf{x}}^{(1)}\right)^T \\ \vdots \\ \left(\bar{\mathbf{x}}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} \bar{x}_1^{(1)} & \bar{x}_2^{(1)} \\ \vdots & \vdots \\ \bar{x}_1^{(151)} & \bar{x}_2^{(151)} \end{bmatrix}$$

Data pre-processing (standardization)

We have an example with 151 points in 2-dimensions $\mathbf{x} \in \mathbb{R}^2$

$$\mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(151)} & x_2^{(151)} \end{bmatrix}, \qquad \mathbf{X} \in \mathbb{R}^{151\times 2}$$

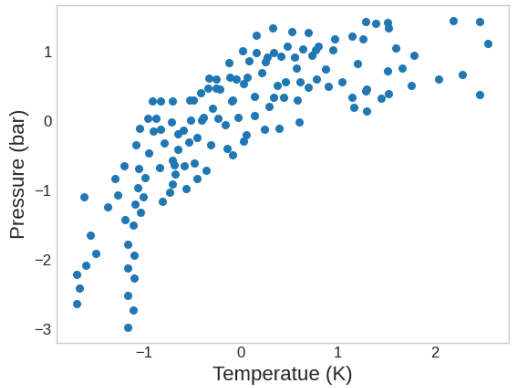
First step for any ML analysis (good practice) is **standardizing** the data:

$$\bar{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

With

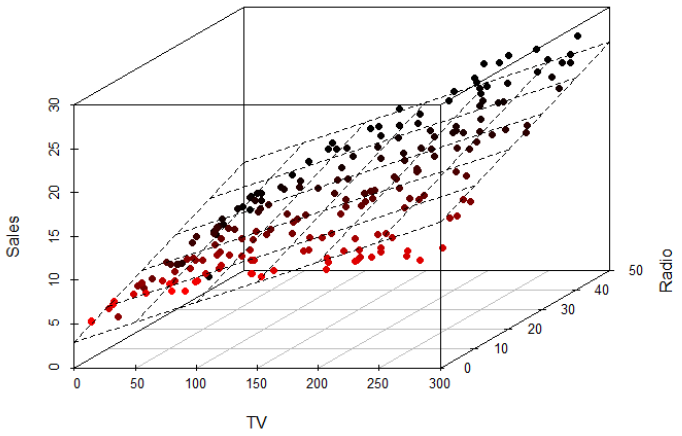
$$\mu_j = \frac{1}{151}\sum_{i=1}^{151} x_j^{(i)}, \qquad \sigma_j = \sqrt{\frac{\sum_{i=1}^{151}\left(x_j^{(i)} - \mu_j\right)^2}{150}}$$

$$\bar{\mathbf{X}} = \begin{bmatrix} \left(\bar{\mathbf{x}}^{(1)}\right)^T \\ \vdots \\ \left(\bar{\mathbf{x}}^{(151)}\right)^T \end{bmatrix} =$$

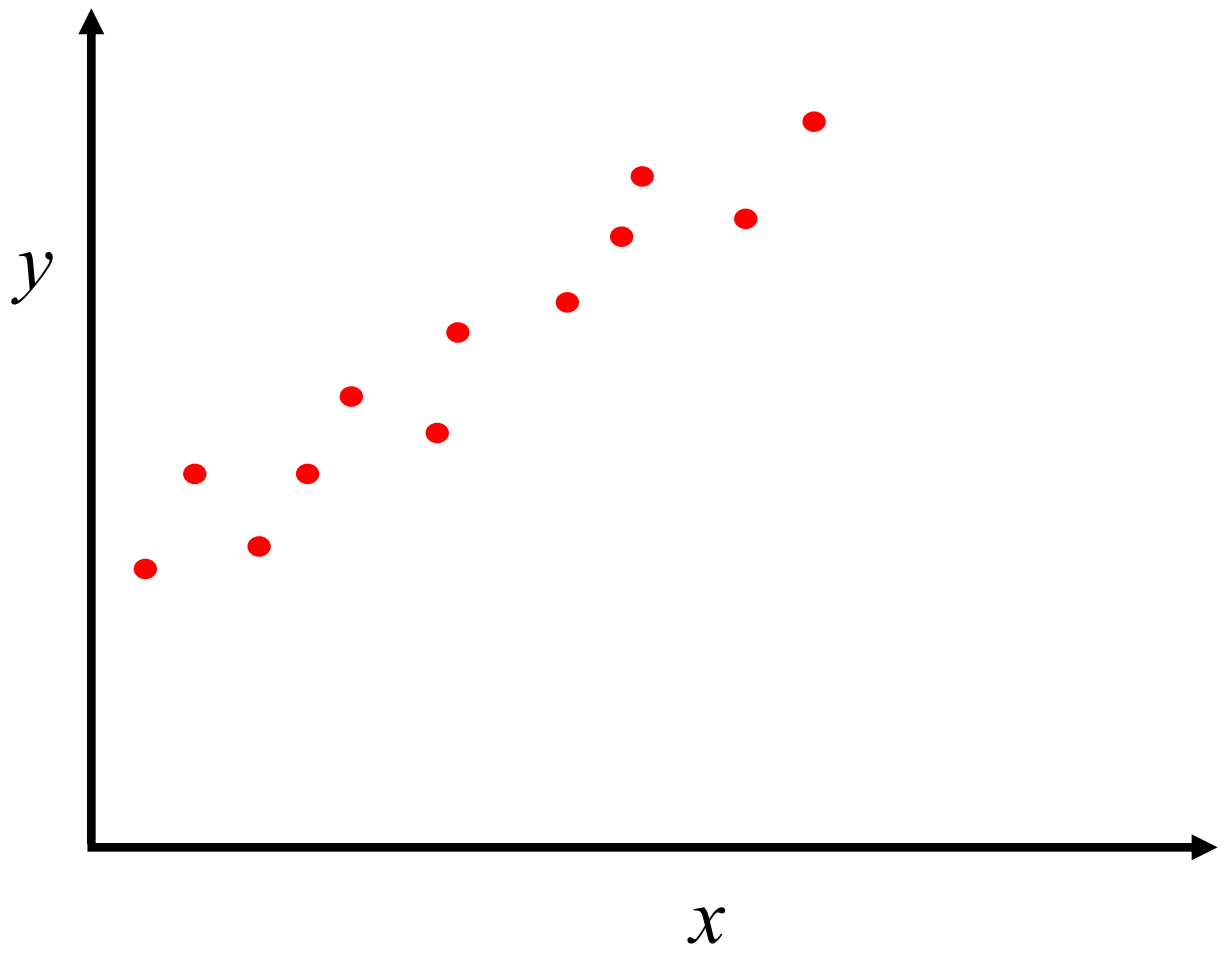


Assume all data is standardized from now on $\mathbf{x} \rightarrow \bar{\mathbf{x}}$

Linear Regression

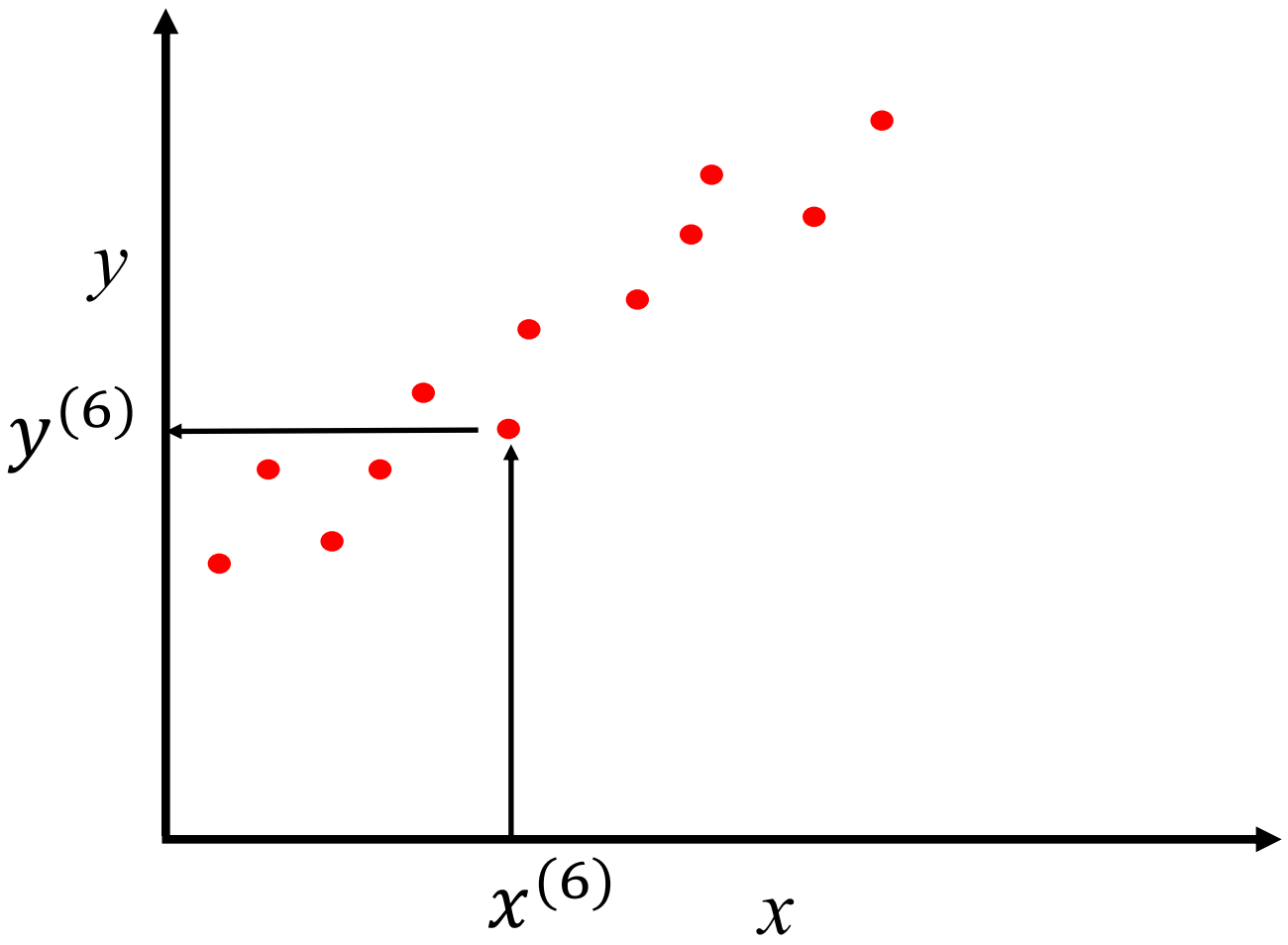


Supervised Learning – Linear Regression 1D example



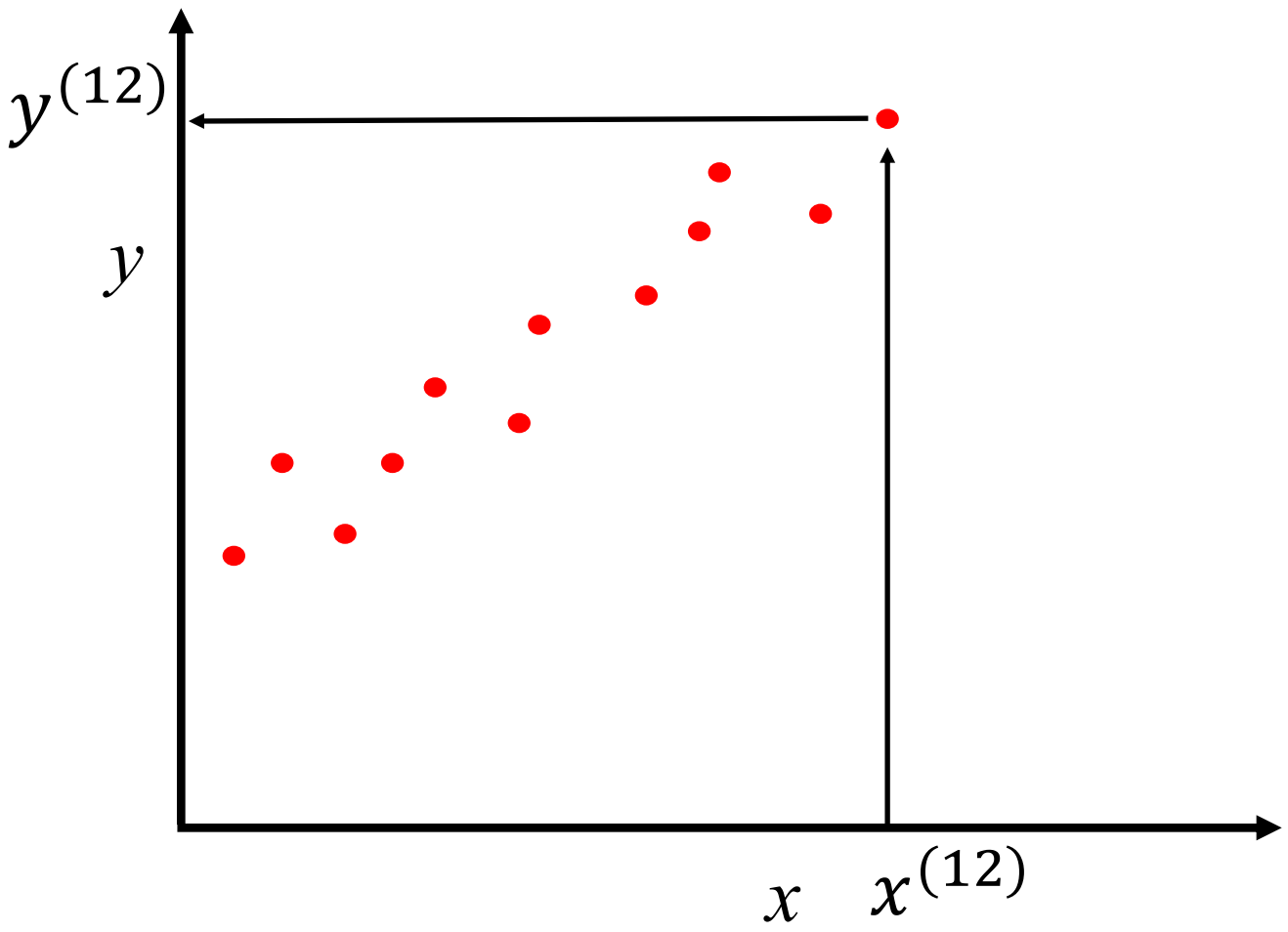
- For several values of x we have their corresponding y value.

Supervised Learning – Linear Regression 1D example



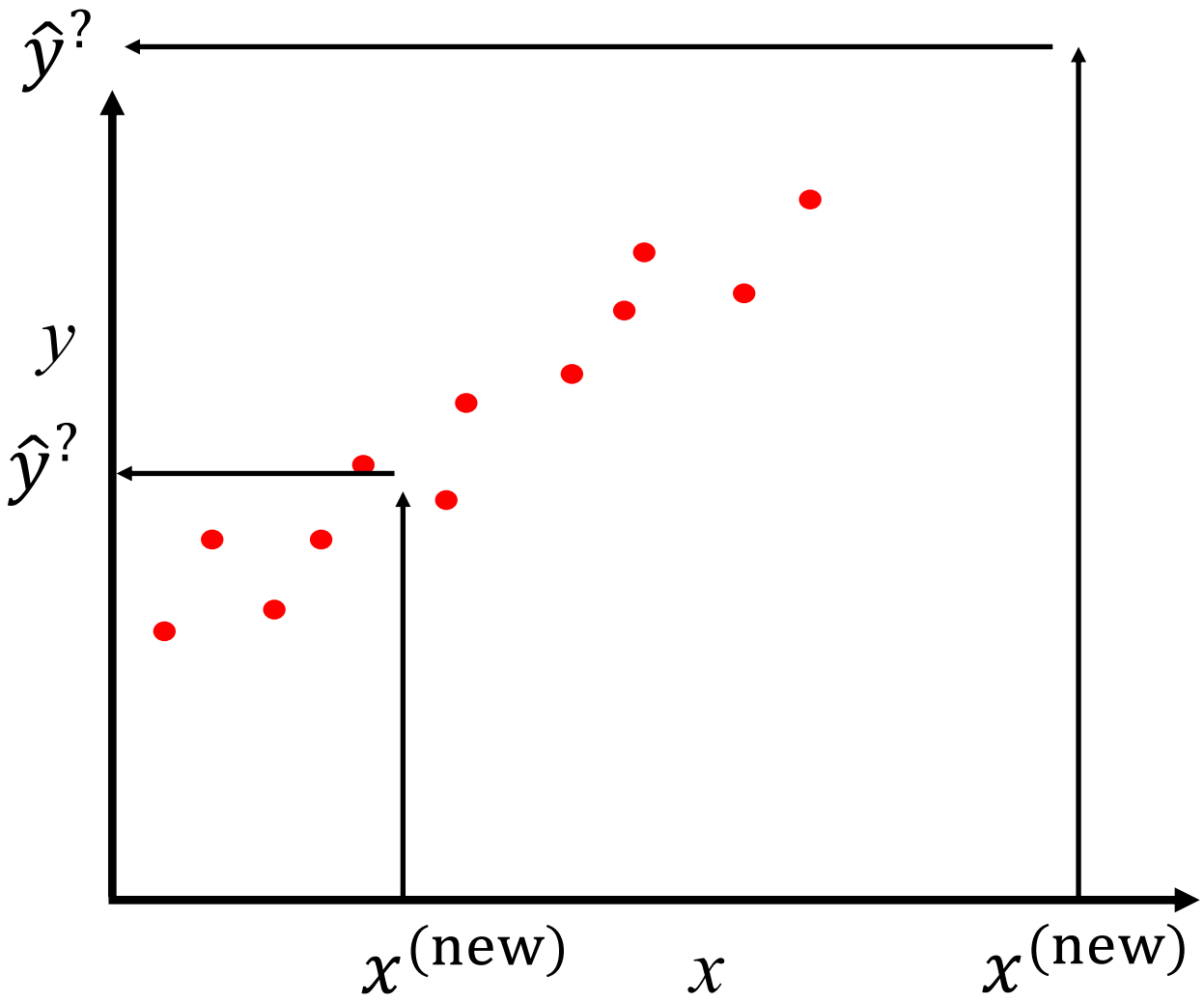
- For several values of x we have their corresponding y value.

Supervised Learning – Linear Regression 1D example



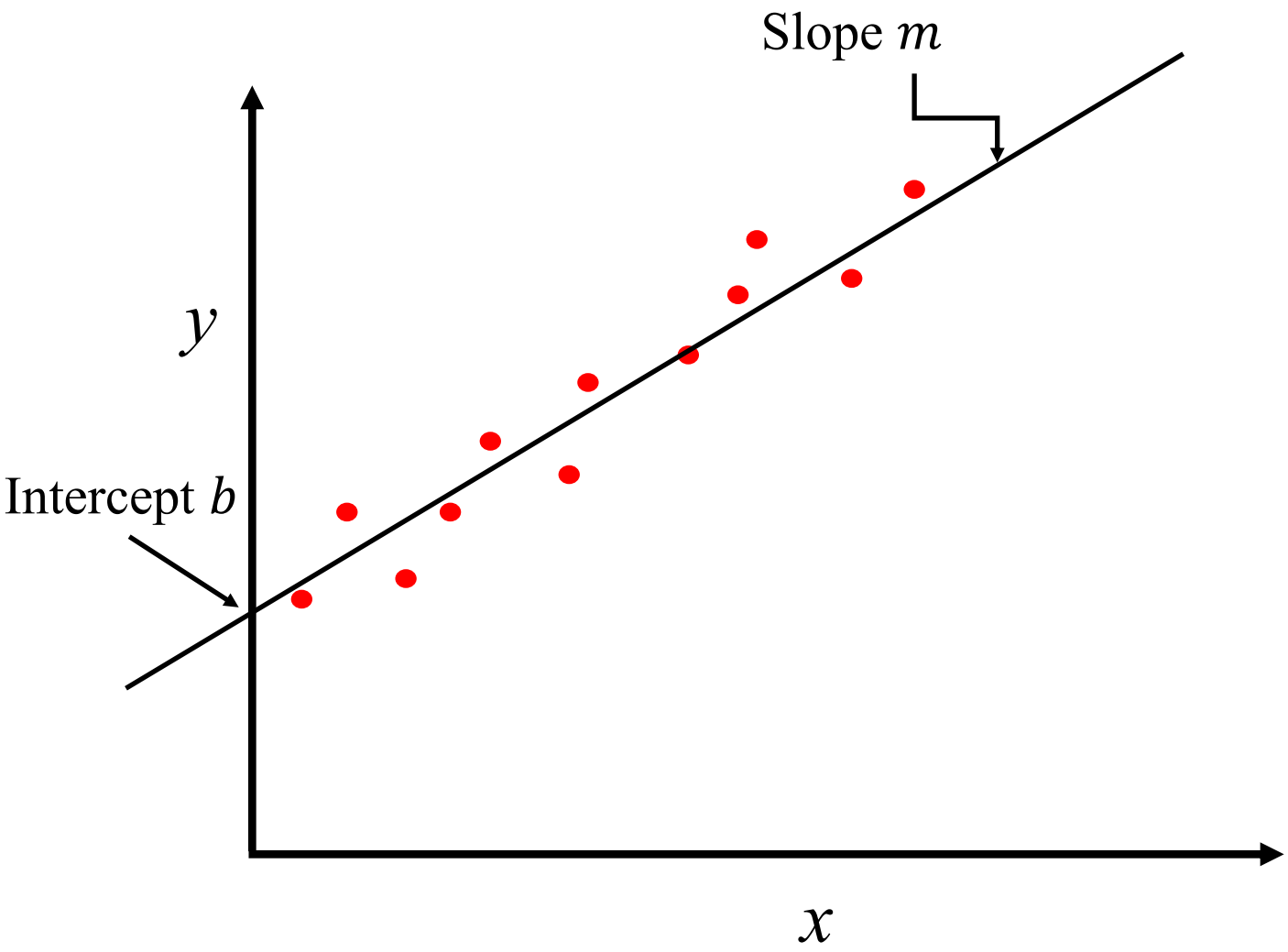
- For several values of x we have their corresponding y value.

Supervised Learning – Linear Regression 1D example



- For several values of x we have their corresponding y value.

Supervised Learning – Linear Regression 1D example



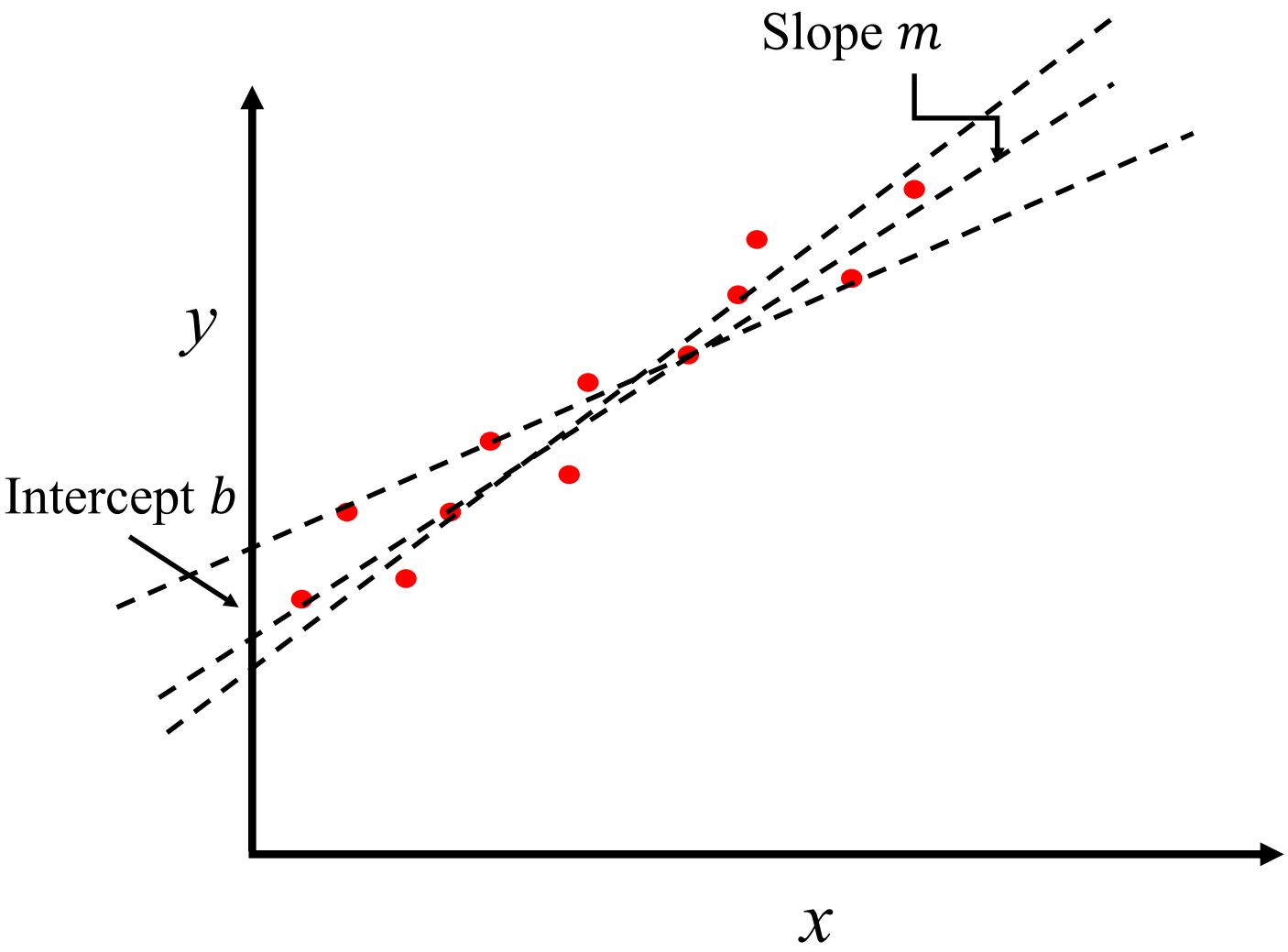
- For several values of x we have their corresponding y value.

- We build a model to predict y for new values of x.

$$\hat{y} = m x + b$$

Parameters to be tuned

Supervised Learning – Linear Regression 1D example

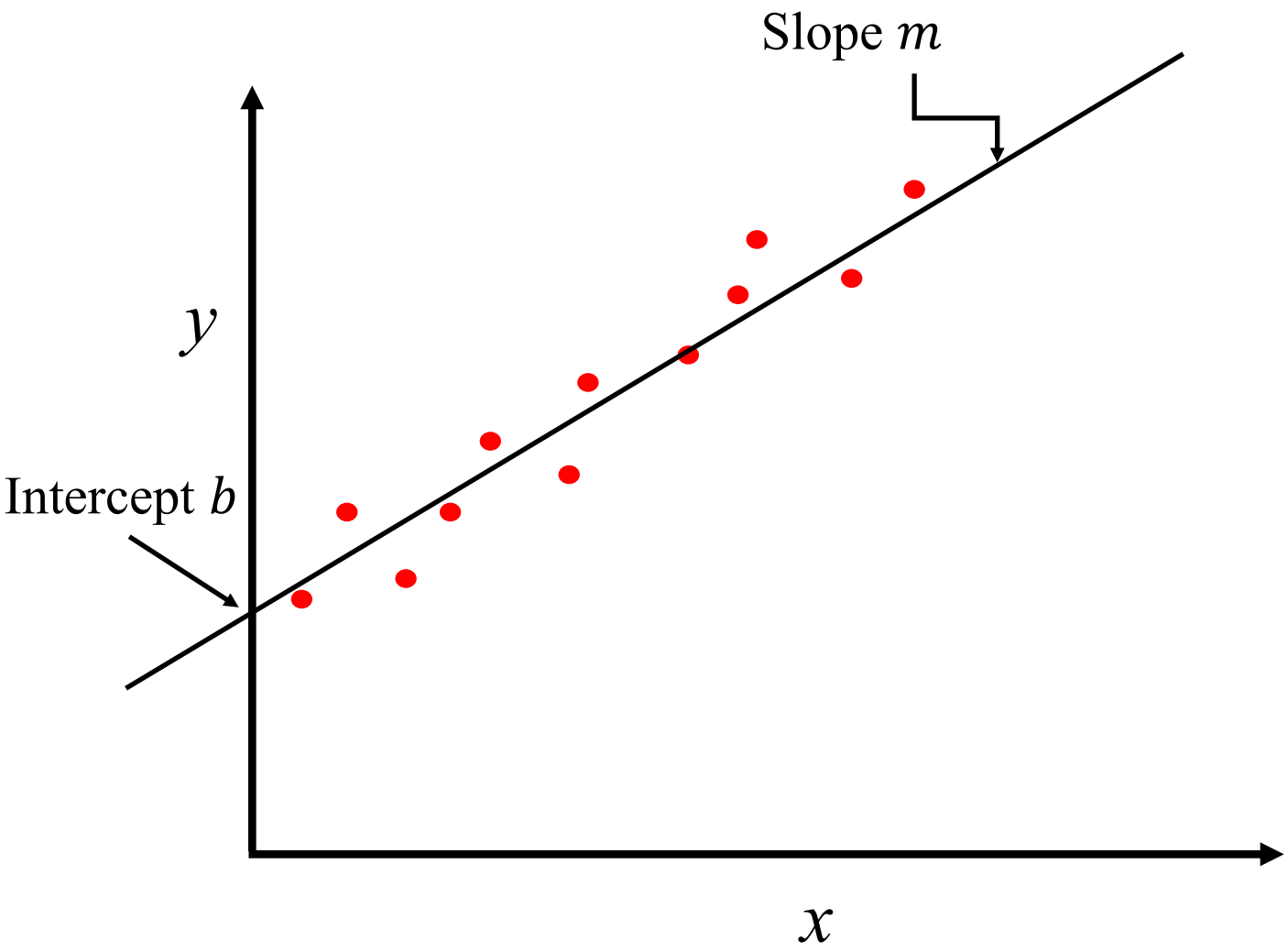


- For several values of x we have their corresponding y value.
- We build a model to predict y for new values of x.

$$\hat{y} = m\,x + b$$

- How do we find the best line (i.e. b and m) that models the data?

Supervised Learning – Linear Regression 1D example



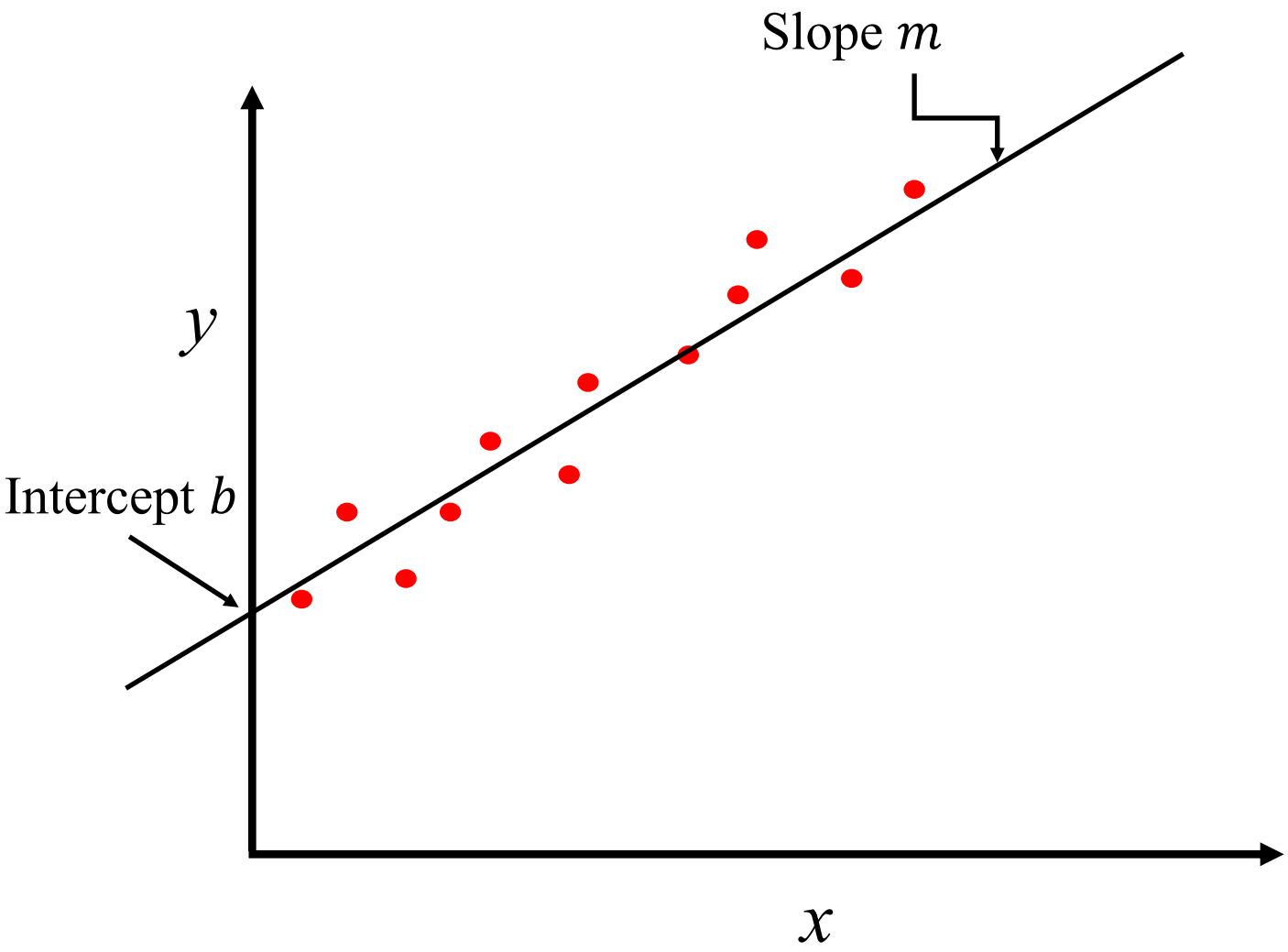
- For several values of x we have their corresponding y value.
- We build a model to predict y for new values of x.

$$\hat{y} = m\,x + b$$

- How do we find the best line (i.e. b and m) that models the data?

Optimization!

Supervised Learning – Linear Regression 1D example

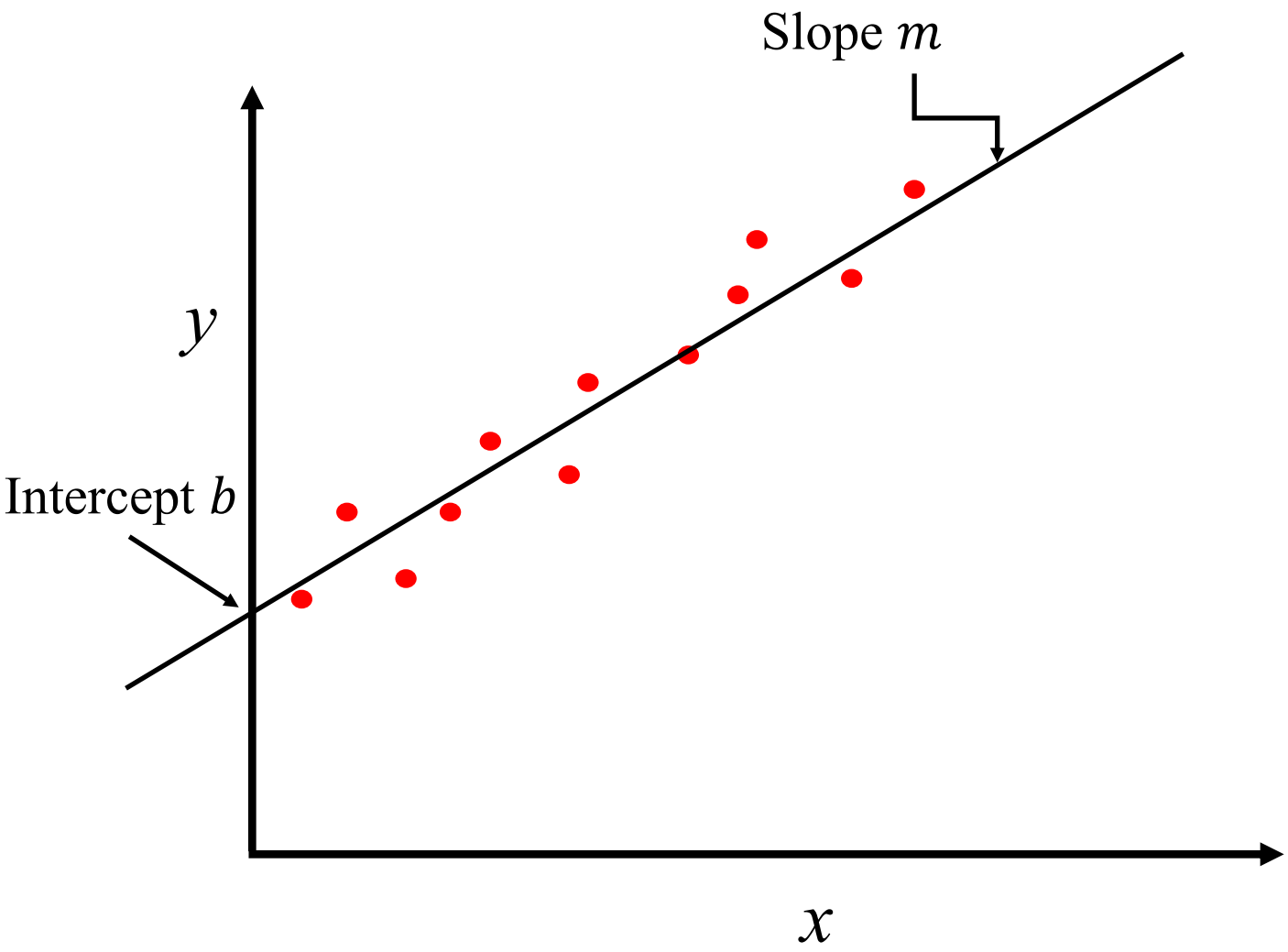


- We wish to find the best values for b and m.

$$\min_{\boxed{m,b}} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

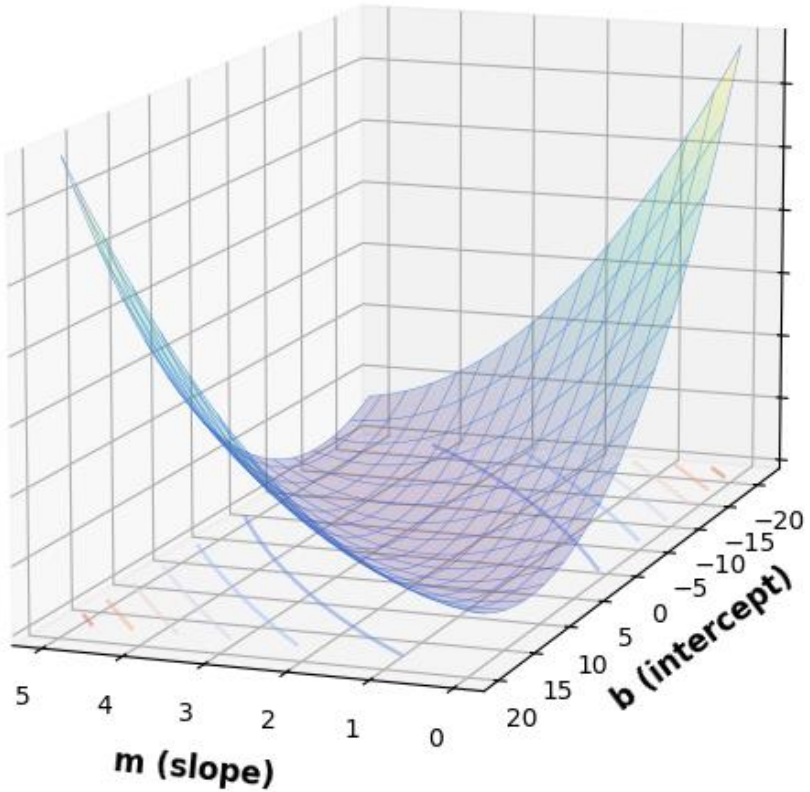
Determine optimal values

Supervised Learning – Linear Regression 1D example

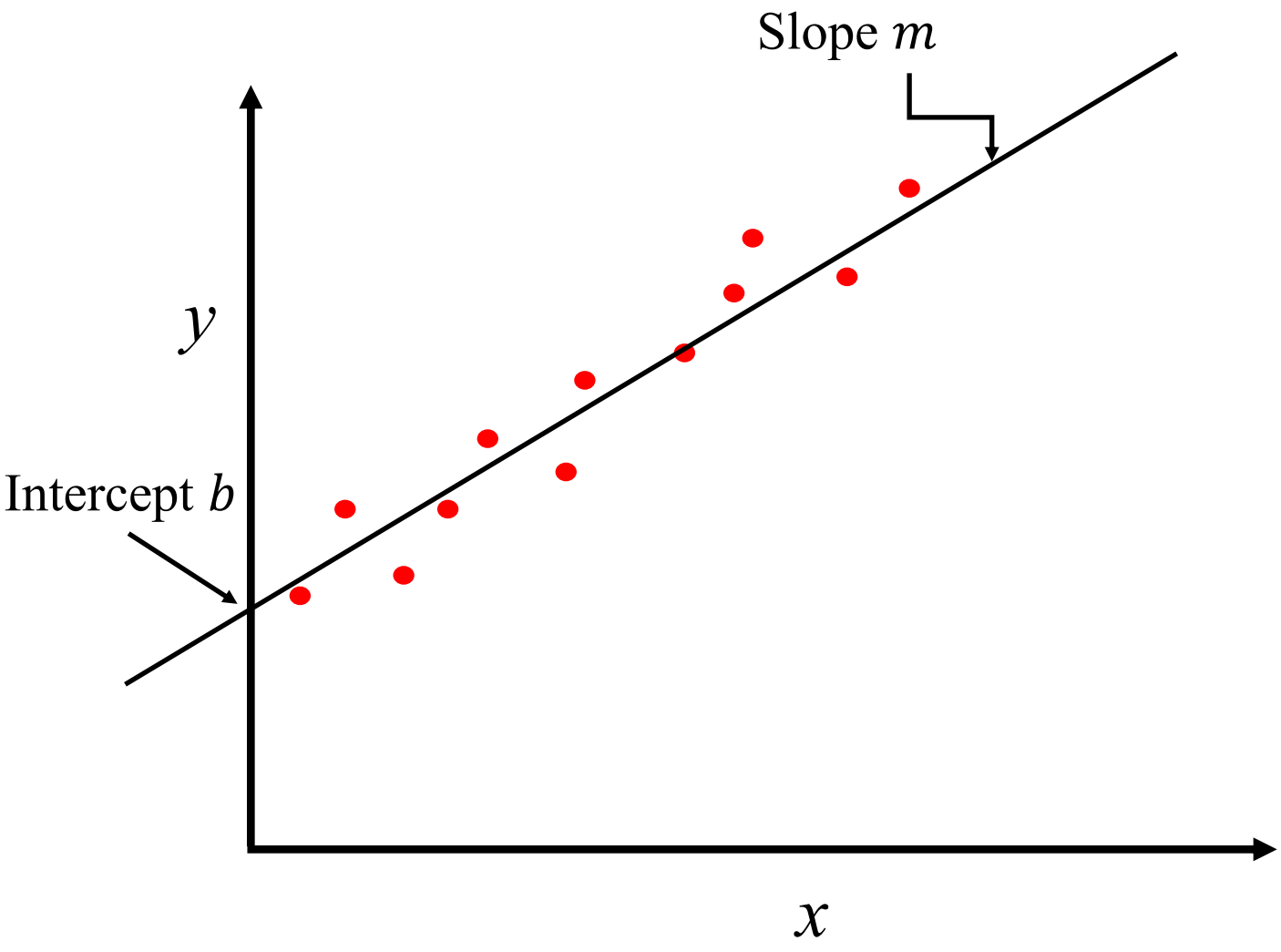


- We wish to find the best values for b and m.

$$\min_{m,b} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$



Supervised Learning – Linear Regression 1D example



- We wish to find the best values for b and m.

$$\min_{\boxed{m,b}} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

Determine optimal values

$$f_{\text{obj}} := \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

Solve:

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial m} = 0, \qquad \frac{\partial f_{\text{obj}}(m,b)}{\partial b} = 0$$

Supervised Learning – Linear Regression 1D example

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial m} = 2\sum_{i=1}^{12} x^{(i)}\left(mx^{(i)} + b - y^{(i)}\right) = 0$$

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial b} = 2\sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right) = 0$$

We can rewrite as:

$$\frac{\partial f_{\text{obj}}}{\partial m} = m\sum_{i=1}^{12} \left(x^{(i)}\right)^2 + b\sum_{i=1}^{12} x^{(i)} - \sum_{i=1}^{12} x^{(i)}y^{(i)} = 0$$

$$\frac{\partial f_{\text{obj}}}{\partial b} = m\sum_{i=1}^{12} x^{(i)} + 12b - \sum_{i=1}^{12} y^{(i)} = 0$$

- We wish to find the best values for b and m.

$$\min_{\boxed{m,b}} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

Determine optimal values

$$f_{\text{obj}} := \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

Solve:

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial m} = 0, \qquad \frac{\partial f_{\text{obj}}(m,b)}{\partial b} = 0$$

Supervised Learning – Linear Regression 1D example

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial m} = 2\sum_{i=1}^{12} x^{(i)}\left(mx^{(i)} + b - y^{(i)}\right) = 0$$

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial b} = 2\sum_{i=1}^{12}\left(mx^{(i)} + b - y^{(i)}\right) = 0$$

We can rewrite as:

$$\frac{\partial f_{\text{obj}}}{\partial m} = m\sum_{i=1}^{12}\left(x^{(i)}\right)^2 + b\sum_{i=1}^{12} x^{(i)} - \sum_{i=1}^{12} x^{(i)}y^{(i)} = 0$$

$$\frac{\partial f_{\text{obj}}}{\partial b} = m\sum_{i=1}^{12} x^{(i)} + 12b - \sum_{i=1}^{12} y^{(i)} = 0$$

- We wish to find the best values for b and m.

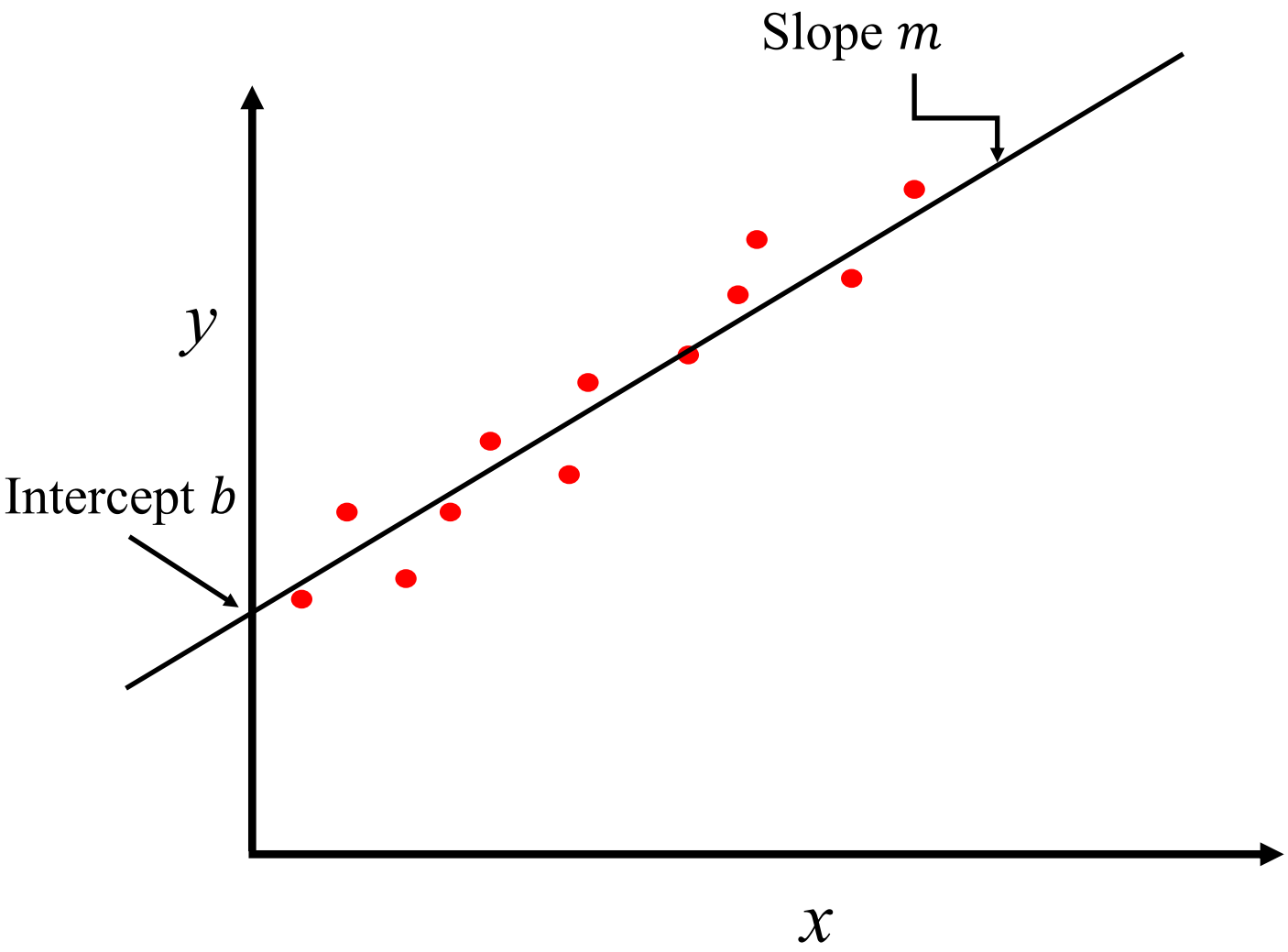
$$\min_{m,b}\sum_{i=1}^{12}\left(mx^{(i)} + b - y^{(i)}\right)^2$$

- Closed form solutions:

$$m = \frac{\sum_{i=1}^{12} x^{(i)}y^{(i)} - \frac{1}{12}\sum_{i=1}^{12} x^{(i)}\sum_{i=1}^{12} y^{(i)}}{\sum_{i=1}^{12}(x^{(i)})^2 - \frac{1}{12}\left(\sum_{i=1}^{12} x^{(i)}\right)^2}$$

$$b = \frac{\sum_{i=1}^{12} y^{(i)} - m\sum_{i=1}^{12} x^{(i)}}{12}$$

Supervised Learning – Linear Regression 1D example



- We wish to find the best values for b and m.

$$\min_{m,b} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

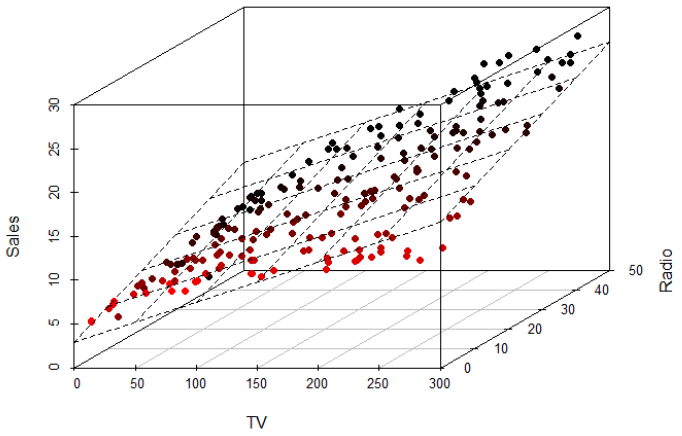
- Closed form solutions:

$$m = \frac{\sum_{i=1}^{12}\left(x^{(i)} - \overline{x}\right)\left(y^{(i)} - \overline{y}\right)}{\sum_{i=1}^{12}\left(x^{(i)} - \overline{x}\right)^2}$$

$$b = \overline{y} - m\overline{x}$$

Where $\overline{x}$ and $\overline{y}$ are the average of the $x^{(i)}$ and $y^{(i)}$, respectively.

Linear regression with multiple inputs

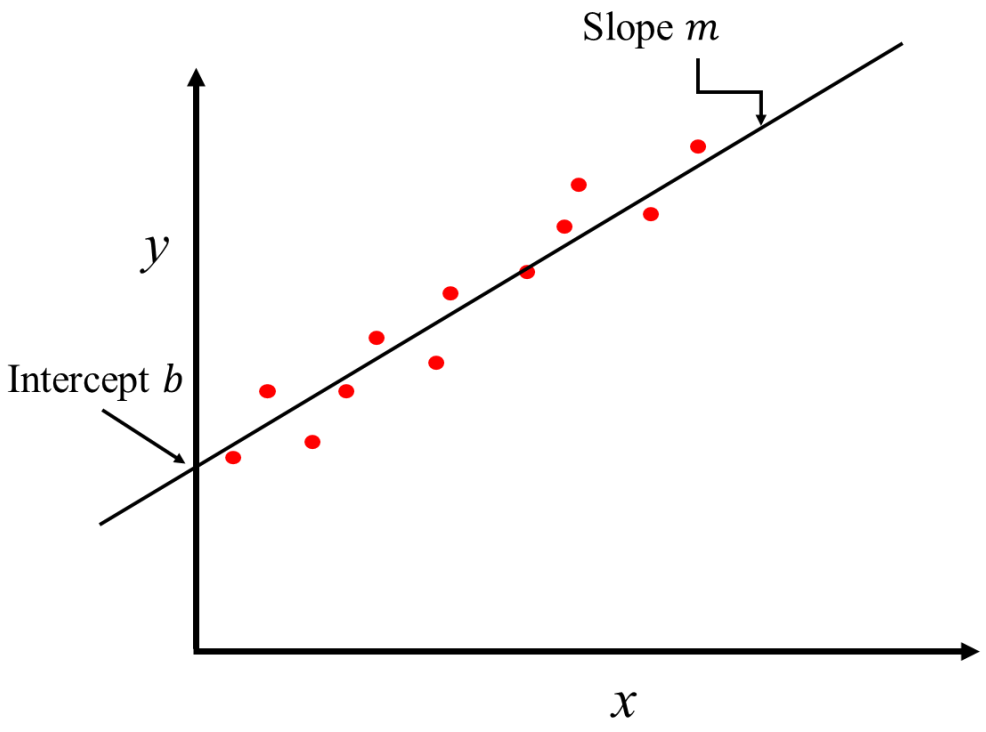


Linear Regression – Multiple Inputs

We went through a 1D example where the input data $x^{(i)} \in \mathbb{R}$ was a scalar

$$\text{Data: } \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n_d)}, y^{(n_d)})\}$$

n_d refers to the number of datapoints.



Linear Regression – Multiple Inputs

We went through a 1D example where the input data $x^{(i)} \in \mathbb{R}$ was a scalar

$$\text{Data: } \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n_d)}, y^{(n_d)})\}$$

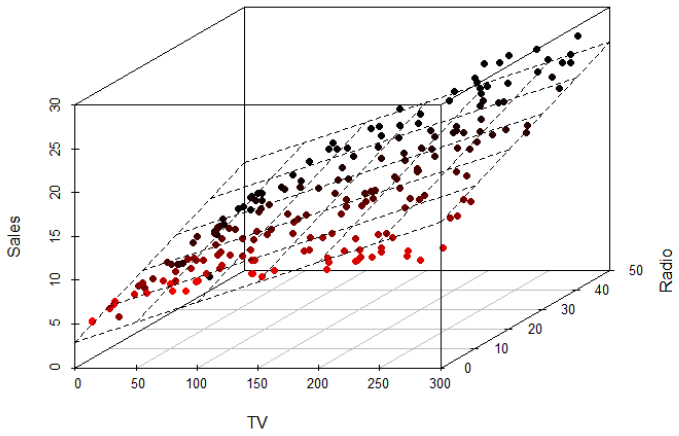
n_d refers to the number of datapoints.

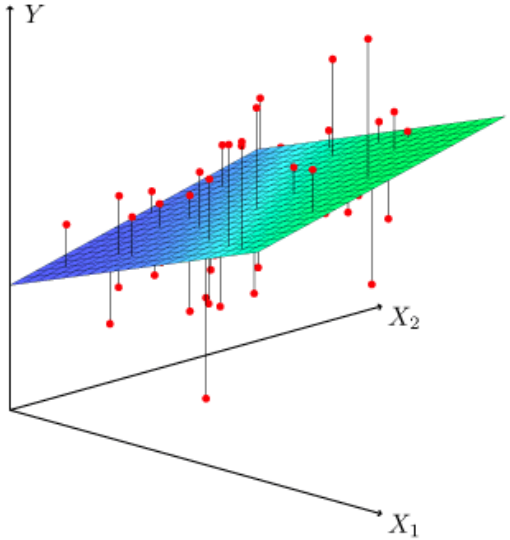
What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

n_x is the dimension of our input.





2D visualization: $n_x = 2$

Linear Regression – Multiple Inputs

What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Now we must determine:

Bias term

$$\hat{y} \approx m_1 x_1 + m_2 x_2 + \cdots + m_{n_x} x_{n_x} + b$$

One "slope"
per dimension

Linear Regression – Multiple Inputs

What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Now we must determine:

$$\hat{y} \approx m_1 x_1 + m_2 x_2 + \cdots + m_{n_x} x_{n_x} + b$$

Change of notation:

$$\hat{y} \approx \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0, \qquad \boldsymbol{\theta} = \left[\theta_0, \theta_1, \dots, \theta_{n_x}\right]^T$$

Linear Regression – Multiple Inputs

What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Change of notation:

$$\hat{y} \approx \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0, \qquad \boldsymbol{\theta} = \left[\theta_0, \theta_1, \dots, \theta_{n_x}\right]^T$$

Now I wish to find the best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \Big(\underbrace{\theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0}_{\text{Model}} - \overbrace{y^{(i)}}^{\text{Data}} \Big)^2$$

Linear Regression – Multiple Inputs

What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Change of notation:

$$\hat{y} \approx \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0, \qquad \boldsymbol{\theta} = \left[\theta_0, \theta_1, \dots, \theta_{n_x}\right]^T$$

Now I wish to find the best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

$$\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \qquad i = 0, \dots n_x$$

<u>Also analytical solution!</u>

Linear Regression – Multiple Inputs

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{bmatrix}$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Linear Regression – Multiple Inputs

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{bmatrix}$$

Solution: $\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Analytical solution!

Linear Regression – Multiple Inputs

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{bmatrix}$$

Solution: $\boldsymbol{\theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\theta}^*$ with $\mathbf{x}_{\text{new}}^T = \left[1, x_1^{(\text{new})}, \dots, x_{n_x}^{(\text{new})}\right]$

Linear Regression – Multiple Inputs

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

$$\boxed{\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix}}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{bmatrix}$$

$$\boxed{\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{n_x} \end{bmatrix}}$$

Solution: $\boldsymbol{\theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\theta}^*$

Linear Regression – Multiple Inputs

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{bmatrix}$$

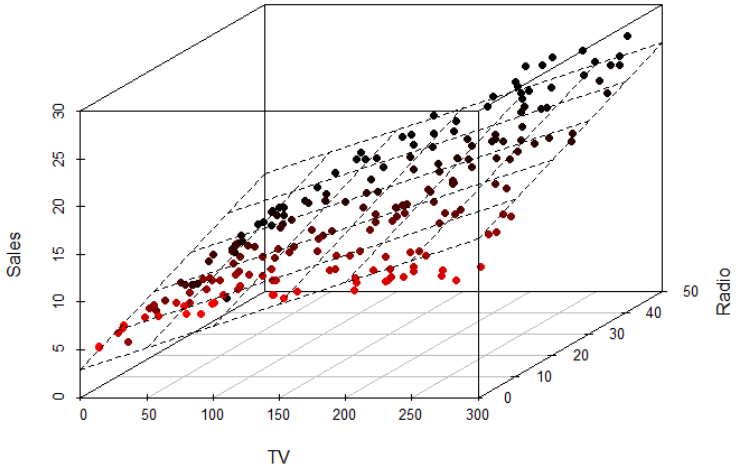
Solution: $\boldsymbol{\theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\theta}^*$

If n_x is very large $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} \approx 0$ is solved by an iterative method.

Deriving the Normal Equation:

$$\boldsymbol{\theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$



Deriving the Normal Equation

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2$$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{bmatrix}$$

Solution: $\boldsymbol{\theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\theta}^*$

Deriving the Normal Equation

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2$$

$$f_{\text{obj}} = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 = (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) = \boldsymbol{\theta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} - \boldsymbol{\theta}^T\mathbf{X}^T\mathbf{y} - \mathbf{y}^T\mathbf{X}\boldsymbol{\theta} + \mathbf{y}^T\mathbf{y}$$

Deriving the Normal Equation

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2$$

$$f_{\text{obj}} = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 = (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) = \boldsymbol{\theta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} - 2(\mathbf{X}^T\mathbf{y})^T\boldsymbol{\theta} + \mathbf{y}^T\mathbf{y}$$

$$\nabla_{\boldsymbol{\theta}} f_{\text{obj}}(\boldsymbol{\theta}) = 2\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} - 2\mathbf{X}^T\mathbf{y}$$

$$2\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} - 2\mathbf{X}^T\mathbf{y} = \mathbf{0} \quad \rightarrow \quad \mathbf{X}^T\mathbf{X}\boldsymbol{\theta} = \mathbf{X}^T\mathbf{y} \quad \rightarrow \quad \boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Deriving the Normal Equation

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2$$

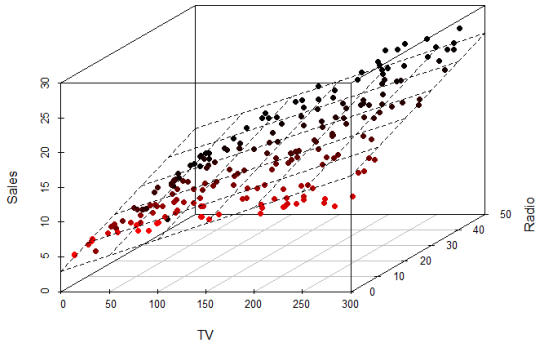
$$f_{\text{obj}} = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 = (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) = \boldsymbol{\theta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} - 2(\mathbf{X}^T\mathbf{y})^T\boldsymbol{\theta} + \mathbf{y}^T\mathbf{y}$$

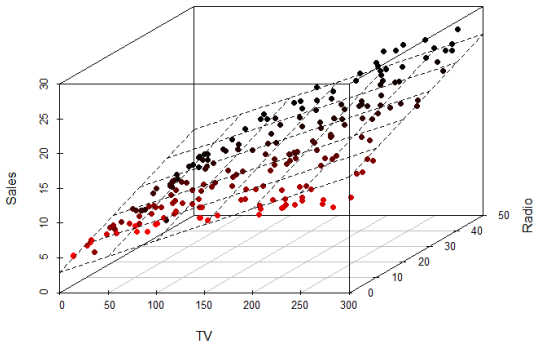
$$\nabla_{\boldsymbol{\theta}} f_{\text{obj}}(\boldsymbol{\theta}) = 2\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} - 2\mathbf{X}^T\mathbf{y}$$

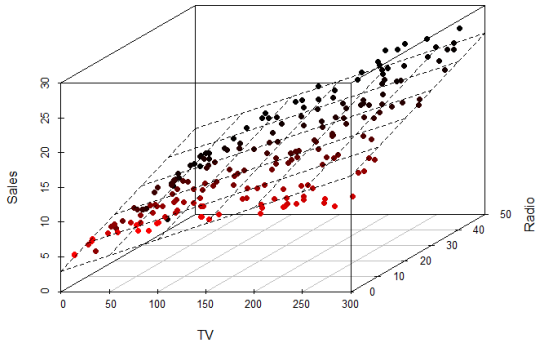
$$\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Remember $m = \frac{\sum_{i=1}^{12}(x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^{12}(x^{(i)} - \bar{x})^2}$

Linear regression with multiple inputs & outputs







Linear Regression - Multiple Inputs & Outputs

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{bmatrix}$$

Solution: $\boldsymbol{\theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\theta}^*$

Linear Regression - Multiple Inputs & Outputs

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$, $\mathbf{y} \in \mathbb{R}^{n_y}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), (\mathbf{x}^{(2)}, \mathbf{y}^{(2)}), \dots, (\mathbf{x}^{(n_d)}, \mathbf{y}^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\Theta}$:

$$\min_{\boldsymbol{\Theta}} \sum_{j=1}^{n_y} \sum_{i=1}^{n_d} \left(\theta_{1,j} x_1^{(i)} + \cdots + \theta_{n_x,j} x_{n_x}^{(i)} + \theta_{0,j} - y_j^{(i)}\right)^2$$

Solution $\frac{\partial f_{\text{obj}}(\boldsymbol{\Theta})}{\partial \theta_{i,j}} = 0, \; i = 0, \dots n_x, \; j = 1, \dots, n_y$

Another bit of notation:

$$y_j^{(i)} \approx \theta_{1,j} x_1^{(i)} + \theta_{2,j} x_2^{(i)} + \cdots + \theta_{n_x,j} x_{n_x}^{(i)} + (1)\theta_{0,j}, \qquad i = 1, \dots, n_d, \qquad j = 1, \dots, n_y$$

$$\boldsymbol{\Theta} = \begin{bmatrix} \boldsymbol{\theta}_0 \\ \boldsymbol{\theta}_1 \\ \vdots \\ \boldsymbol{\theta}_{n_x} \end{bmatrix}, \qquad \mathbf{Y} = \begin{bmatrix} \left(\mathbf{y}^{(1)}\right)^T \\ \left(\mathbf{y}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{y}^{(n_d)}\right)^T \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{bmatrix}$$

Solution: $\boldsymbol{\Theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$ New observation: $\mathbf{y}_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\Theta}^*$

Code example

```python
#####################
# Linear Regression #
#####################

class Linear_Regression_model:

    def __init__(self):
        """Initialize the model. Compute the coefficients (theta)  later."""
        self.theta = None

    def fit(self, X, y):
        """
        Trains the model using the Normal Equation:
        """
        n_d         = X.shape[0]
        X_with_bias = np.hstack([np.ones((n_d, 1)), X])

```

Linear Regression – Code example

$$\mathbf{X} = \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n_d)} \end{bmatrix},$$

```
1 Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}$$

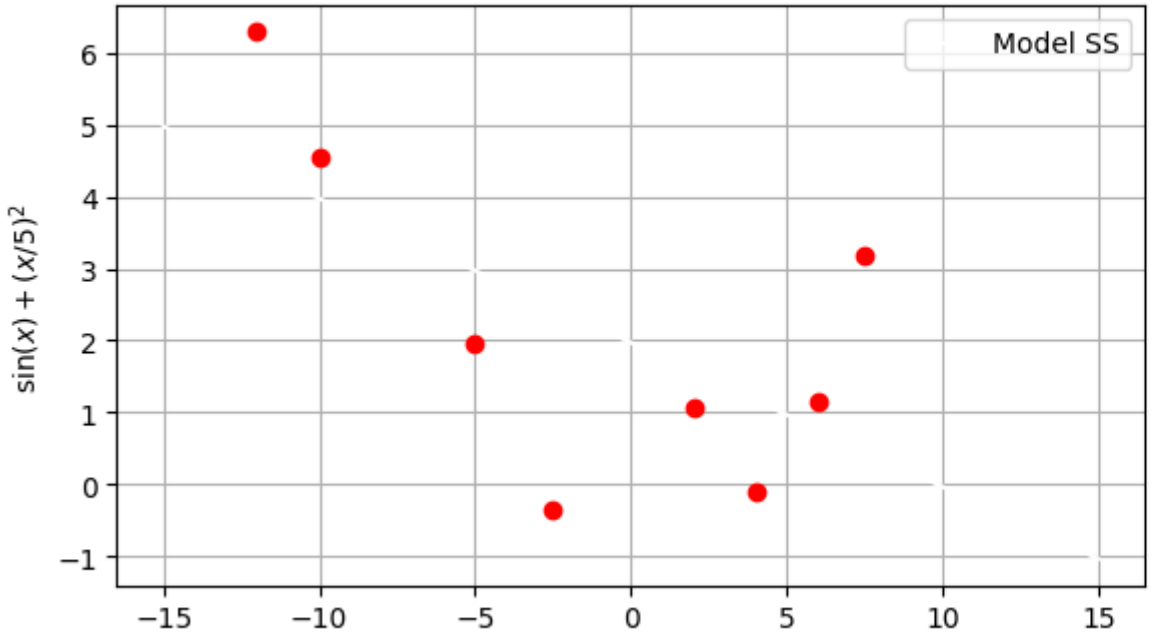
Linear Regres

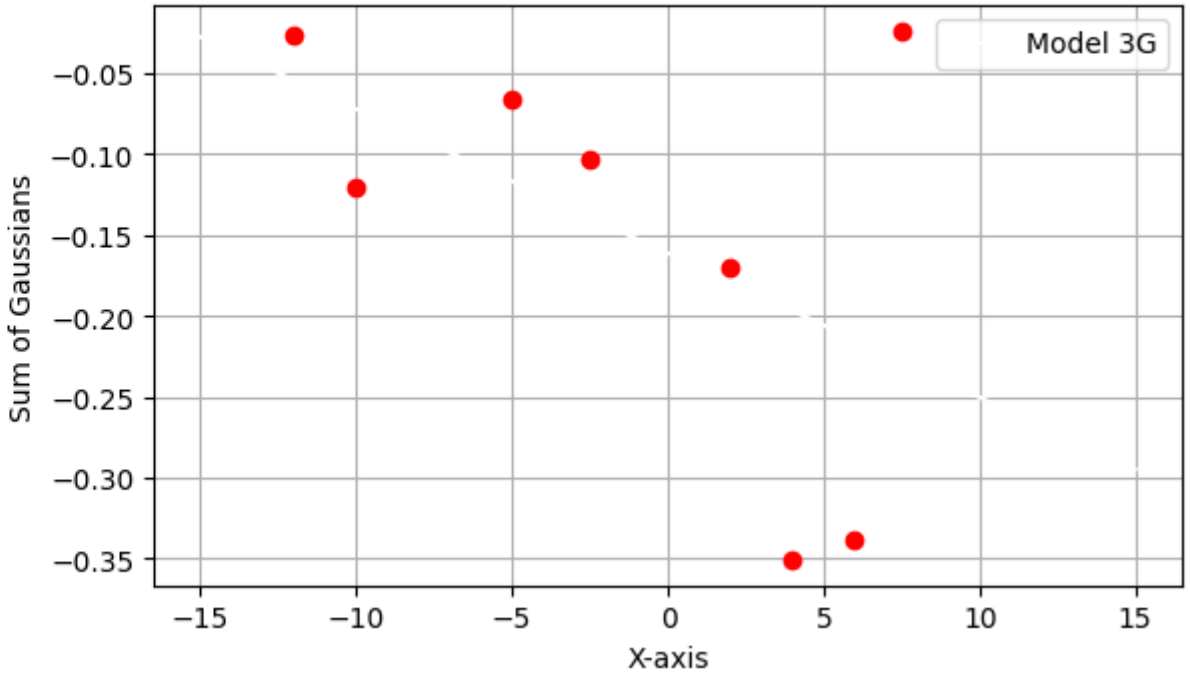
```
1 Ytrain_sin_square
```

```
array([[ 6.3  ],
       [ 4.54 ],
       [ 1.96 ],
       [-0.348],
       [ 1.07 ],
       [-0.117],
       [ 1.16 ],
       [ 3.19 ]])
```

```
1 Ytrain_3Gaussians
```

```
array([[-0.027 ],
       [-0.121 ],
       [-0.0663],
       [-0.104 ],
       [-0.17  ],
       [-0.35  ],
       [-0.339 ],
       [-0.0245]])
```





Linear Regression – Code example

```python
#####################
# Linear Regression #
#####################

class Linear_Regression_model:

    def __init__(self, X, y):

        n_d         = X.shape[0]
        X_with_bias = np.hstack([np.ones((n_d, 1)), X])

        # We use @ for matrix multiplication
        XT_X = X_with_bias.T @ X_with_bias
        XT_y = X_with_bias.T @ y
        I    = np.eye(XT_X.shape[0])

        # theta = (X^T * X)^-1 * X^T * y
        self.theta = np.linalg.inv(XT_X + 1e-6 * I) @ XT_y

    ######################
    # --- Prediction --- #
    ######################

    def predict(self, x_new):
        # add bias
        x_new_with_bias = np.hstack([np.ones((1, 1)), x_new])

        return x_new_with_bias @ self.theta
```

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} \\ 1 & x_1^{(2)} \\ & \vdots \\ 1 & x_1^{(n_d)} \end{bmatrix}$$

Note: $\texttt{np.ones} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}$

```python
Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

```python
Ytrain_sin_square
```

```
array([[ 6.3   ],
       [ 4.54  ],
       [ 1.96  ],
       [-0.348],
       [ 1.07  ],
       [-0.117],
       [ 1.16  ],
       [ 3.19  ]])
```

Linear Regression – Code example

```python
#####################
# Linear Regression #
#####################

class Linear_Regression_model:

    def __init__(self, X, y):

        n_d         = X.shape[0]
        X_with_bias = np.hstack([np.ones((n_d, 1)), X])

        # We use @ for matrix multiplication
        XT_X = X_with_bias.T @ X_with_bias
        XT_y = X_with_bias.T @ y
        I    = np.eye(XT_X.shape[0])

        # theta = (X^T * X)^-1 * X^T * y
        self.theta = np.linalg.inv(XT_X + 1e-6 * I) @ XT_y

    ######################
    # --- Prediction --- #
    ######################

    def predict(self, x_new):
        # add bias
        x_new_with_bias = np.hstack([np.ones((1, 1)), x_new])

        return x_new_with_bias @ self.theta
```

$\mathbf{X}^T\mathbf{X}$

$\mathbf{X}^T\mathbf{y}$

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} \\ 1 & x_1^{(2)} \\ & \vdots \\ 1 & x_1^{(n_d)} \end{bmatrix}$$

```
1 Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

```
1 Ytrain_sin_square
```

```
array([[ 6.3   ],
       [ 4.54  ],
       [ 1.96  ],
       [-0.348],
       [ 1.07  ],
       [-0.117],
       [ 1.16  ],
       [ 3.19  ]])
```

Solution: $\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Note:
`@ for matrix multiplication`

$$\texttt{np.eye} = \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix}$$

Linear Regression – Code example

```python
#####################
# Linear Regression #
#####################

class Linear_Regression_model:

    def __init__(self, X, y):

        n_d         = X.shape[0]
        X_with_bias = np.hstack([np.ones((n_d, 1)), X])

        # We use @ for matrix multiplication
        XT_X = X_with_bias.T @ X_with_bias
        XT_y = X_with_bias.T @ y
        I    = np.eye(XT_X.shape[0])

        # theta = (X^T * X)^-1 * X^T * y
        self.theta = np.linalg.inv(XT_X + 1e-6 * I) @ XT_y

    ######################
    # --- Prediction --- #
    ######################

    def predict(self, x_new):
        # add bias
        x_new_with_bias = np.hstack([np.ones((1, 1)), x_new])

        return x_new_with_bias @ self.theta
```

$\mathbf{X}^T\mathbf{X}$

$\mathbf{X}^T\mathbf{y}$

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} \\ 1 & x_1^{(2)} \\ & \vdots \\ 1 & x_1^{(n_d)} \end{bmatrix}$$

```
1 Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

```
1 Ytrain_sin_square
```

```
array([[ 6.3   ],
       [ 4.54  ],
       [ 1.96  ],
       [-0.348],
       [ 1.07  ],
       [-0.117],
       [ 1.16  ],
       [ 3.19  ]])
```

Solution: $\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X} + \varepsilon\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$

Note:
`@ for matrix multiplication`

$$\texttt{np.eye} = \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix}$$

Linear Regression – Code example

```python
#####################
# Linear Regression #
#####################

class Linear_Regression_model:

    def __init__(self, X, y):

        n_d         = X.shape[0]
        X_with_bias = np.hstack([np.ones((n_d, 1)), X])

        # We use @ for matrix multiplication
        XT_X = X_with_bias.T @ X_with_bias
        XT_y = X_with_bias.T @ y
        I    = np.eye(XT_X.shape[0])

        # theta = (X^T * X)^-1 * X^T * y
        self.theta = np.linalg.inv(XT_X + 1e-6 * I) @ XT_y

    ######################
    # --- Prediction --- #
    ######################

    def predict(self, x_new):
        # add bias
        x_new_with_bias = np.hstack([np.ones((1, 1)), x_new])

        return x_new_with_bias @ self.theta
```

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} \\ 1 & x_1^{(2)} \\ & \vdots \\ 1 & x_1^{(n_d)} \end{bmatrix}$$

$\mathbf{X}^T\mathbf{X}$

$\mathbf{X}^T\mathbf{y}$

Solution: $\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X} + \varepsilon\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$

New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T\boldsymbol{\theta}$

```
Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

```
Ytrain_sin_square
```

```
array([[ 6.3   ],
       [ 4.54  ],
       [ 1.96  ],
       [-0.348],
       [ 1.07  ],
       [-0.117],
       [ 1.16  ],
       [ 3.19  ]])
```

Linear Regression – Code example

$$\mathbf{X} = \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n_d)} \end{bmatrix},$$

```
1 Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}$$

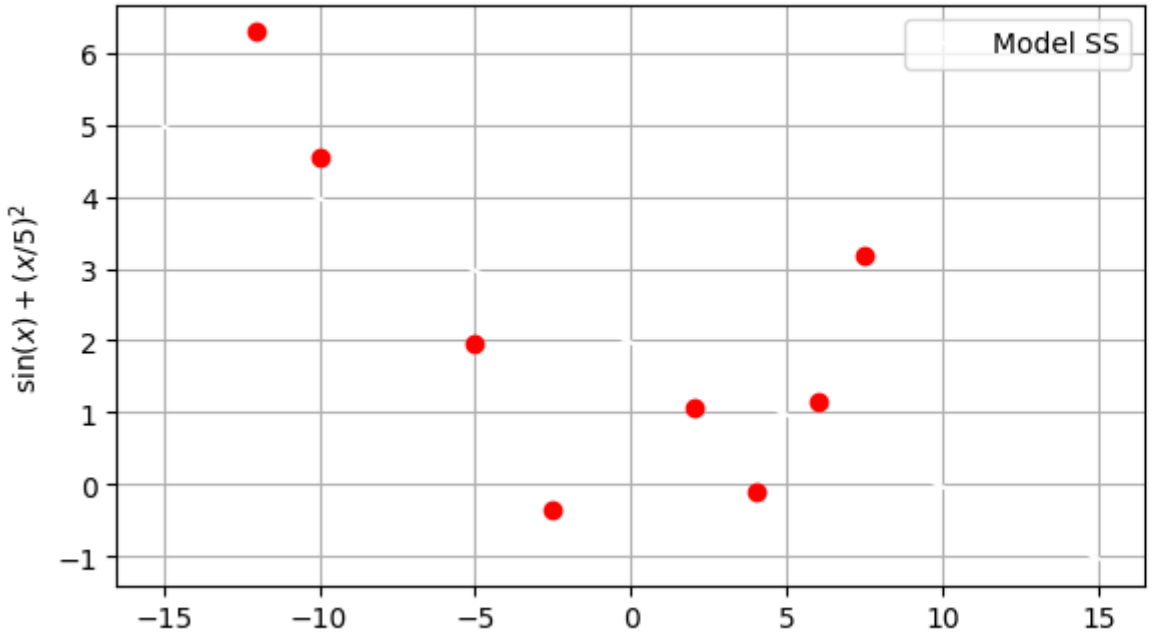
Linear Regres

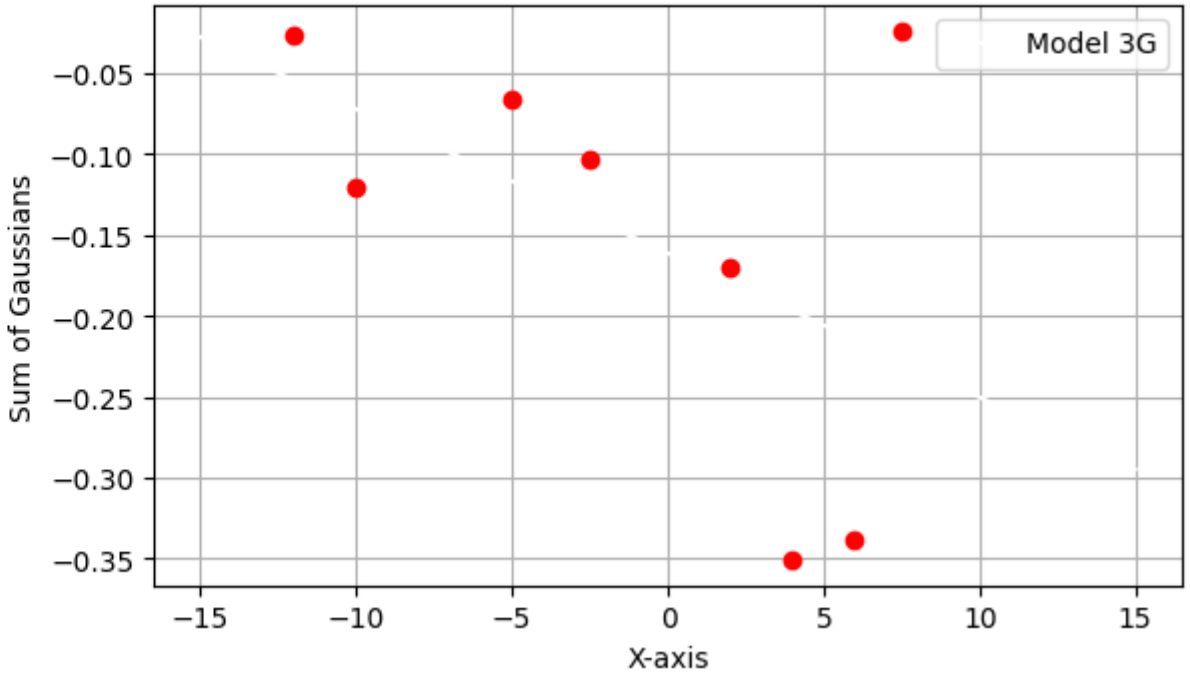
```
1 Ytrain_sin_square
```

```
array([[ 6.3  ],
       [ 4.54 ],
       [ 1.96 ],
       [-0.348],
       [ 1.07 ],
       [-0.117],
       [ 1.16 ],
       [ 3.19 ]])
```

```
1 Ytrain_3Gaussians
```

```
array([[-0.027 ],
       [-0.121 ],
       [-0.0663],
       [-0.104 ],
       [-0.17  ],
       [-0.35  ],
       [-0.339 ],
       [-0.0245]])
```





Linear Regression – Code example

$$\mathbf{X} = \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n_d)} \end{bmatrix},$$

```
1 Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}$$

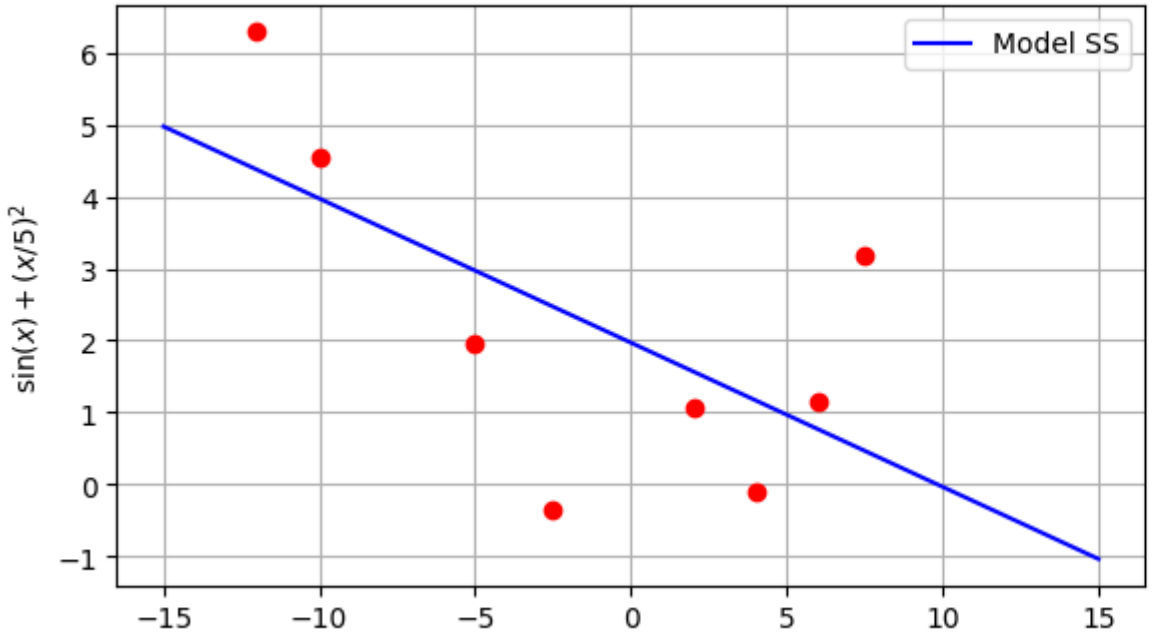
```
1 Ytrain_sin_square
```

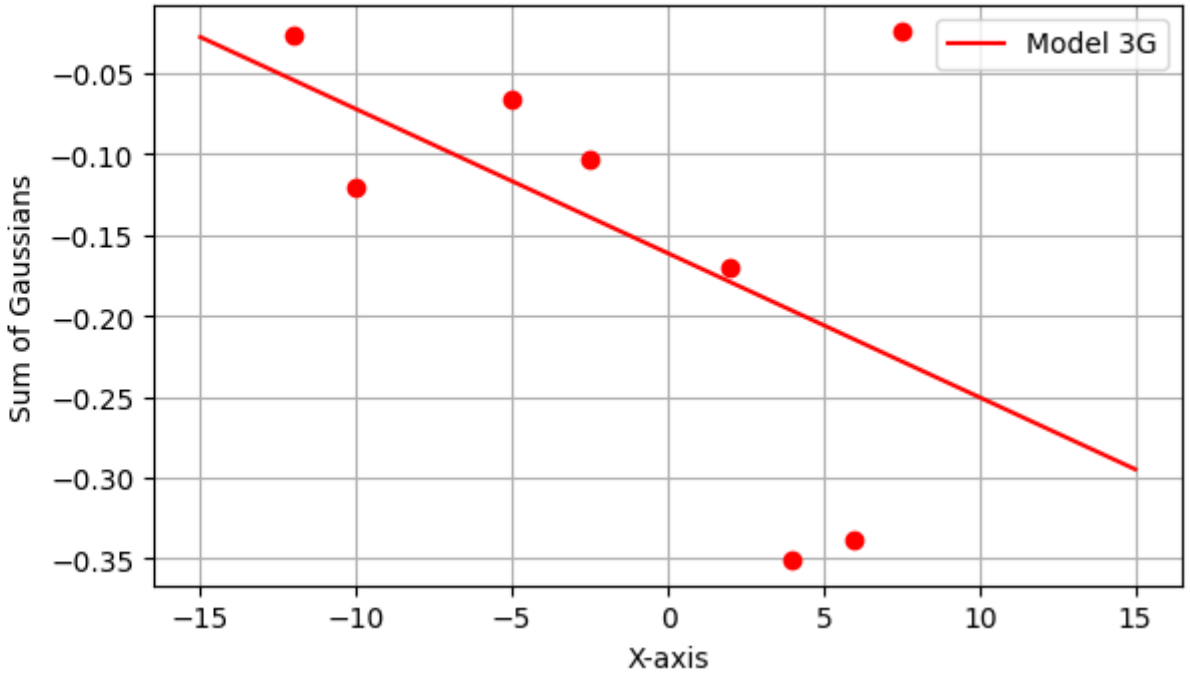
```
array([[ 6.3  ],
       [ 4.54 ],
       [ 1.96 ],
       [-0.348],
       [ 1.07 ],
       [-0.117],
       [ 1.16 ],
       [ 3.19 ]])
```

```
1 Ytrain_3Gaussians
```

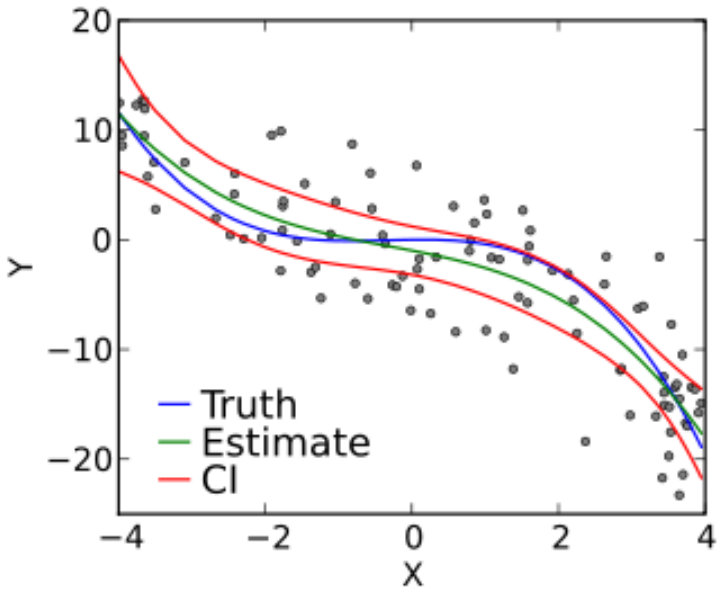
```
array([[-0.027 ],
       [-0.121 ],
       [-0.0663],
       [-0.104 ],
       [-0.17  ],
       [-0.35  ],
       [-0.339 ],
       [-0.0245]])
```

Linear Regres





General Linear Regression



Supervised Learning – General Linear Regression

Capturing nonlinear trends in our data with **linear regression:**

Currently:

$$y \approx \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0$$

If we know trends are <u>quadratic</u> we could do:

$$y \approx \theta_1 x_1 + \theta_{1,1} x_1^2 + \theta_{1,2} x_1 x_2 + \theta_2 x_2 + \theta_{2,2} x_2^2 + \cdots + \theta_{n_x,n_x} x_{n_x}^2 + \theta_0$$

You can make many combinations or even functions:

$$y \approx \theta_1 \phi_1(\mathbf{x}) + \cdots + \theta_{n_\phi} \phi_{n_\phi}(\mathbf{x}) + \theta_0$$

Practically any function of your vector $\mathbf{x}$, e.g.:

$$\phi(\mathbf{x}) \coloneqq \sin(\mathbf{x}), \exp(\mathbf{x}), \log(\mathbf{x}), \ldots$$

Supervised Learning – General Linear Regression

```python
##############################
# Basis Function Regression #
##############################

class BasisF_Regression_model:

    def __init__(self, X, y):
        n_samples = X.shape[0]

        # Basis expansion
        X_squared = X ** 2

        X_cubed   = X ** 3

        X_sin     = np.sin(X)

        X_cos     = np.cos(X)

        bias      = np.ones((n_samples, 1))

        # Combine all features
        X_basis = np.hstack([bias, X, X_squared, X_cubed, X_sin, X_cos ])

        XT_X = X_basis.T @ X_basis
        XT_y = X_basis.T @ y
        I    = np.eye(XT_X.shape[0])

        self.theta = np.linalg.inv(XT_X + 1e-6 * I) @ XT_y
```

$\phi(\mathbf{x}) := x^2$

$\phi(\mathbf{x}) := x^3$

$\phi(\mathbf{x}) := \sin(x)$

$\phi(\mathbf{x}) := \cos(x)$

```python
Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

```python
Ytrain_sin_square
```

```
array([[ 6.3  ],
       [ 4.54 ],
       [ 1.96 ],
       [-0.348],
       [ 1.07 ],
       [-0.117],
       [ 1.16 ],
       [ 3.19 ]])
```

Supervised Learning – General Linear Regression

```python
#############################
# Basis Function Regression #
#############################

class BasisF_Regression_model:

    def __init__(self, X, y):
        n_samples = X.shape[0]

        # Basis expansion
        X_squared = X ** 2

        X_cubed   = X ** 3

        X_sin     = np.sin(X)

        X_cos     = np.cos(X)

        bias      = np.ones((n_samples, 1))

        # Combine all features
        X_basis = np.hstack([bias, X, X_squared, X_cubed, X_sin, X_cos ])

        XT_X = X_basis.T @ X_basis
        XT_y = X_basis.T @ y
        I    = np.eye(XT_X.shape[0])

        self.theta = np.linalg.inv(XT_X + 1e-6 * I) @ XT_y
```

$\phi(\mathbf{x}) := x^2$

$\phi(\mathbf{x}) := x^3$

$\phi(\mathbf{x}) := \sin(x)$

$\phi(\mathbf{x}) := \cos(x)$

$\boldsymbol{\phi}(\mathbf{x}) := [1, x, x^2, x^3, \sin(x), \cos(x)]$

$\mathbf{X}_{\mathrm{bf}}^T \mathbf{X}_{\mathrm{bf}}$

$\mathbf{X}_{\mathrm{bf}}^T \mathbf{y}$

$$\mathbf{X}_{\mathrm{bf}} = \begin{bmatrix} \boldsymbol{\phi}\left(x^{(1)}\right)^T \\ \boldsymbol{\phi}\left(x^{(2)}\right)^T \\ \vdots \\ \boldsymbol{\phi}\left(x^{(n_d)}\right)^T \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}$$

$$\mathbf{X}_{\mathrm{bf}} = \begin{bmatrix} 1, x^{(1)}, \left(x^{(1)}\right)^2, \left(x^{(1)}\right)^3, \sin\left(x^{(1)}\right), \cos\left(x^{(1)}\right) \\ 1, x^{(2)}, \left(x^{(2)}\right)^2, \left(x^{(2)}\right)^3, \sin\left(x^{(2)}\right), \cos\left(x^{(2)}\right) \\ \vdots \\ 1, x^{(n_d)}, \left(x^{(n_d)}\right)^2, \left(x^{(n_d)}\right)^3, \sin\left(x^{(n_d)}\right), \cos\left(x^{(n_d)}\right) \end{bmatrix}$$

Solution: $\boldsymbol{\theta} = \left(\mathbf{X}_{\mathrm{bf}}^T \mathbf{X}_{\mathrm{bf}} + \varepsilon \mathbf{I}\right)^{-1} \mathbf{X}_{\mathrm{bf}}^T \mathbf{y}$

Supervised Learning – General Linear Regression

$$\mathbf{X}_{\mathrm{bf}} = \begin{bmatrix} \boldsymbol{\phi}\left(x^{(1)}\right)^T \\ \boldsymbol{\phi}\left(x^{(2)}\right)^T \\ \vdots \\ \boldsymbol{\phi}\left(x^{(n_d)}\right)^T \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}$$

```
1 Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

```
1 Ytrain_sin_square
```

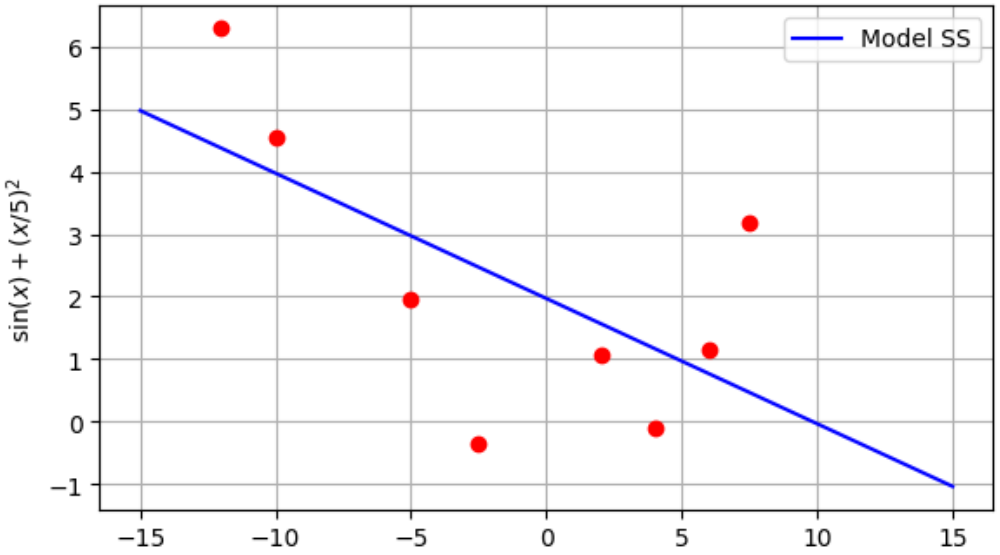
```
array([[ 6.3  ],
       [ 4.54 ],
       [ 1.96 ],
       [-0.348],
       [ 1.07 ],
       [-0.117],
       [ 1.16 ],
       [ 3.19 ]])
```

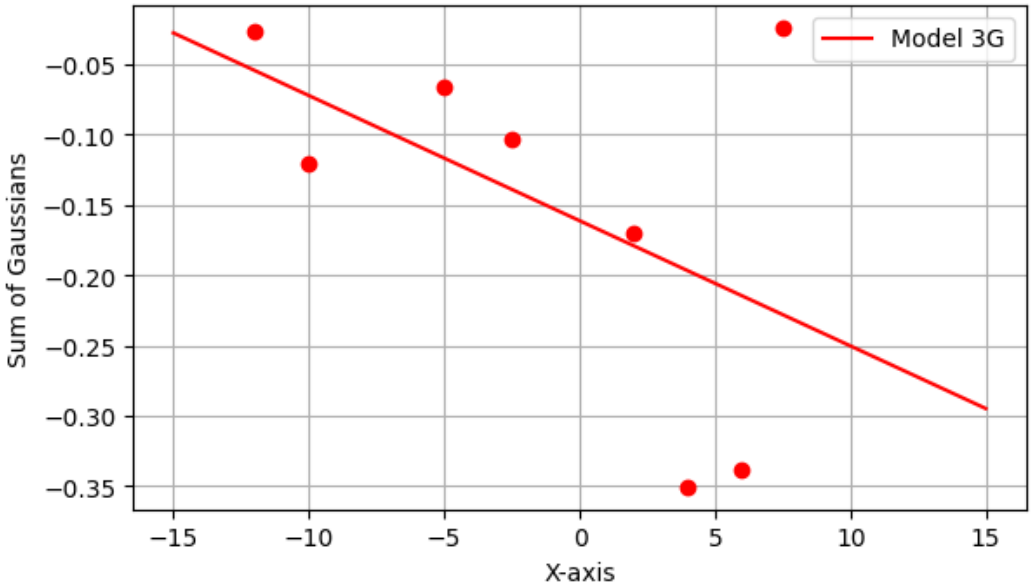
```
1 Ytrain_3Gaussians
```

```
array([[-0.027 ],
       [-0.121 ],
       [-0.0663],
       [-0.104 ],
       [-0.17  ],
       [-0.35  ],
       [-0.339 ],
       [-0.0245]])
```

$$\boldsymbol{\phi}(\mathbf{x}) := [1, x, x^2, x^3, \sin(x), \cos(x)]^T$$

Linear Regression





Supervised Learning – General Linear Regression

$$\mathbf{X}_{\text{bf}} = \begin{bmatrix} \boldsymbol{\phi}\left(x^{(1)}\right)^T \\ \boldsymbol{\phi}\left(x^{(2)}\right)^T \\ \vdots \\ \boldsymbol{\phi}\left(x^{(n_d)}\right)^T \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{bmatrix}$$

```
1 Xtrain
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

```
1 Ytrain_sin_square
```

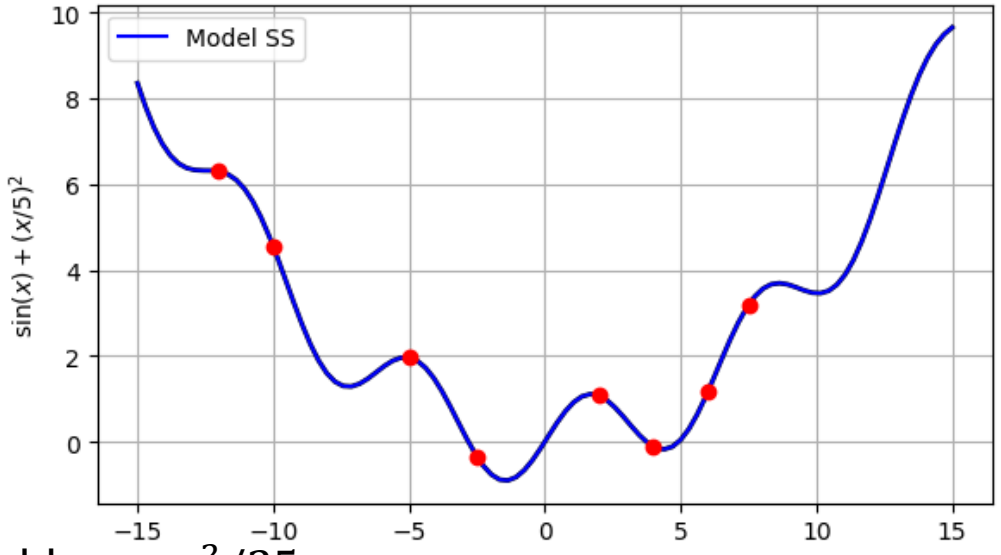
```
array([[ 6.3  ],
       [ 4.54 ],
       [ 1.96 ],
       [-0.348],
       [ 1.07 ],
       [-0.117],
       [ 1.16 ],
       [ 3.19 ]])
```

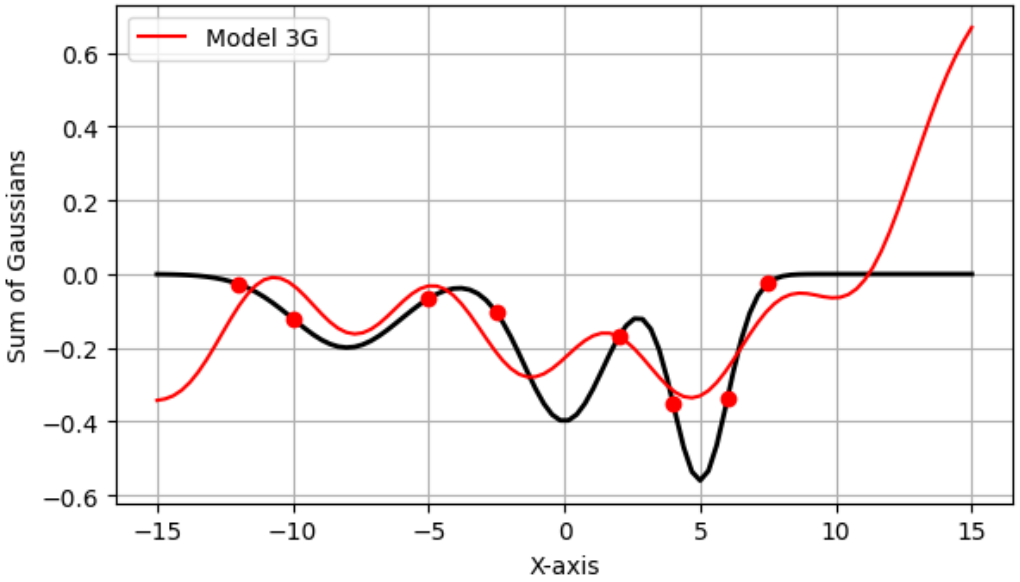
```
1 Ytrain_3Gaussians
```

```
array([[-0.027 ],
       [-0.121 ],
       [-0.0663],
       [-0.104 ],
       [-0.17  ],
       [-0.35  ],
       [-0.339 ],
       [-0.0245]])
```

$$\boldsymbol{\phi}(\mathbf{x}) := [1, x, x^2, x^3, \sin(x), \cos(x)]^T$$

Basis Functions Regression

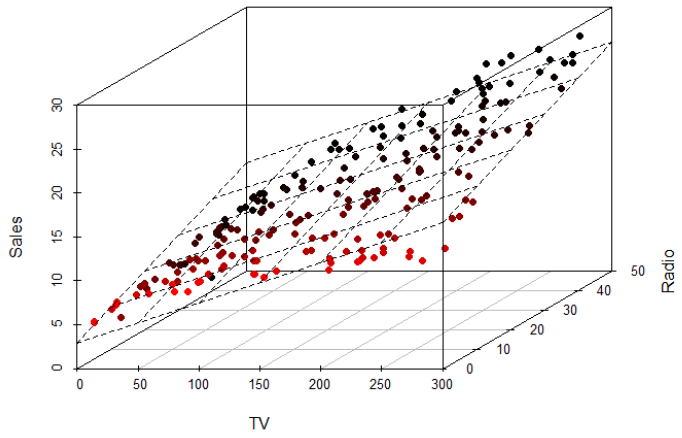




*Here I should say: $x^2/25$

Summary – linear regression

Data: $\{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$



Least Squares

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

$$\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \qquad i = 0, \dots n_x$$

Matrix form:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$$

Fitted values

$$\hat{\mathbf{y}} = \theta_0 + \sum_{i=1}^{n_x} \theta_i x_i\,, \qquad \text{or,} \qquad \hat{\mathbf{y}} = \mathbf{x}^T \boldsymbol{\theta}$$

Note: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0$ is solved by an iterative procedure if $\boldsymbol{\theta}$ or $\mathbf{X}$ is too large.

For nonlinear functions we can:

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^{n_d} \left(\theta_1 \phi_1(\mathbf{x}) + \cdots + \theta_{n_\phi} \phi_{n_\phi}(\mathbf{x}) + \theta_0 - y^{(i)}\right)^2$$

$$\hat{\mathbf{y}} = \theta_0 + \sum_{i=1}^{n_\phi} \theta_i \phi_i(\mathbf{x})$$

IMPERIAL

Department of Chemical Engineering

Machine Learning for Chemical Engineering

Lecture 1c – Machine Learning Models